

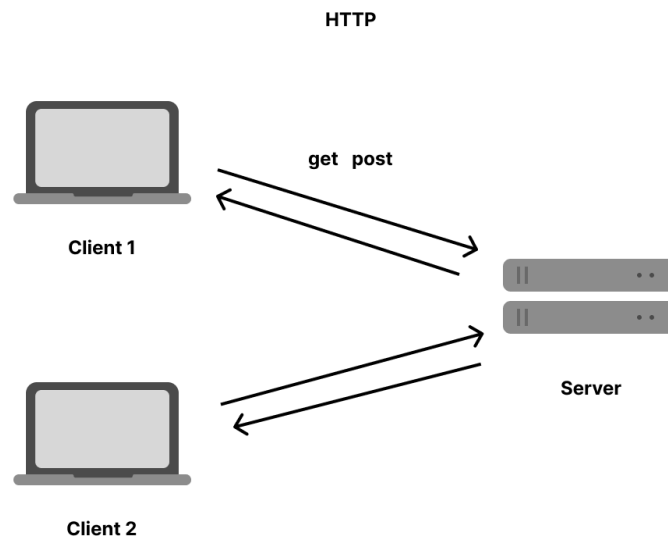
How to Make Messages Live

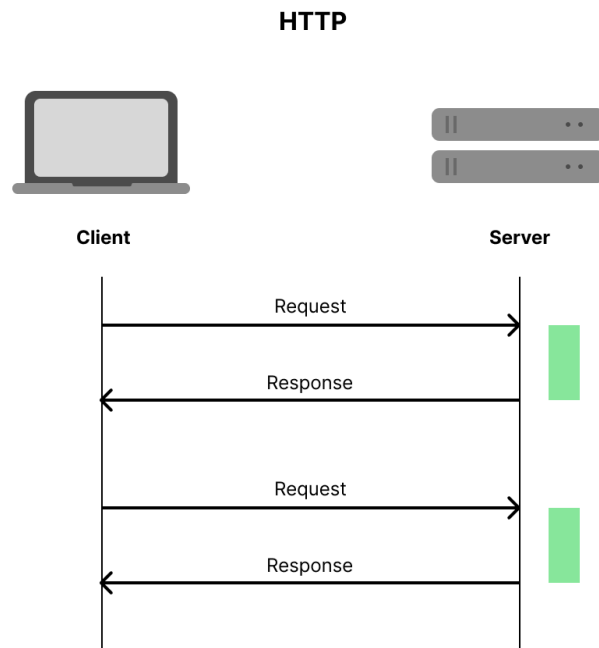
Have you ever built a chat app where users had to keep refreshing to see new messages? That's because traditional HTTP just isn't made for live conversations.

In this blog, we're going to explore how we can transform a basic message system into a **real-time chat** using different techniques. We'll start with the problem, try short polling and long polling, and finally reach the hero of live messaging: **WebSocket**.

The Problem with Traditional HTTP Requests

HTTP follows a simple pattern: the client sends a request, the server responds, and the connection closes.





A diagram showing client sending a GET/POST and receiving a response

This model works well for static pages or one-time queries, but in real-time systems like chat, it falls short.

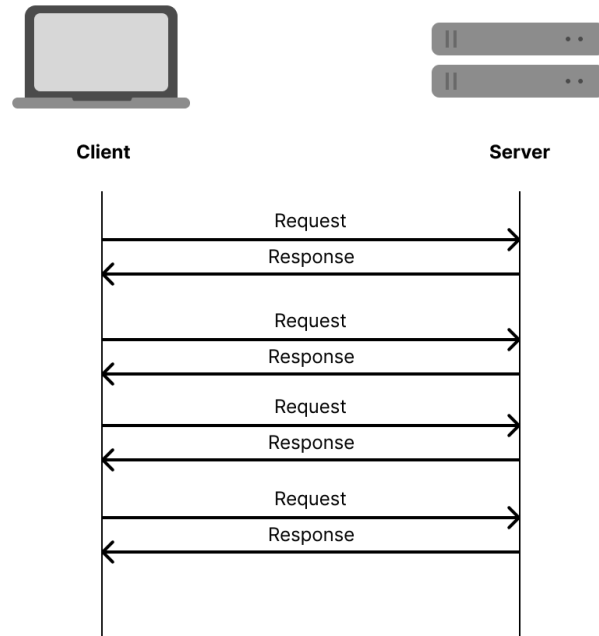
Let's say you send a message to your friend. Until they manually ask the server by making a GET request, they won't know you sent anything.

To "simulate" live updates, many beginners end up calling the server repeatedly, which brings us to...

1. Short Polling

Short polling is a quick hack: your client sends a GET request to the server every few seconds (e.g., using `setInterval()` in JavaScript). It keeps asking, "Do I have new messages?"

HTTP Short polling



HTTP Short Polling request/response diagram

Short Polling server Node.js code

```
const express = require("express");
const cors = require("cors");
const path = require("path");
const app = express();
app.use(cors());
app.use(express.json());

let messages = [];

app.get('/', (req, res) => {
  res.sendFile(path.join(__dirname, 'views', 'index.html'));
});

app.get("/messages", (req, res) => {
  res.json(messages);
});

app.post("/messages", (req, res) => {
```

```

    const { text } = req.body;
    messages.push({ text, timestamp: Date.now() });
    res.sendStatus(200);
  });

  app.listen(3001, () => console.log("Short Polling server running on port 3001"));

```

Short Polling client HTML code

```

<!DOCTYPE html>
<html>
<body>
  <h3>Short Polling Chat</h3>
  <input id="msg" placeholder="Type a message" />
  <button onclick="send()">Send</button>
  <ul id="chat"></ul>

  <script>
    const url = "http://localhost:3001";
    const chat = document.getElementById("chat");

    async function fetchMessages() {
      const res = await fetch(`${url}/messages`);
      const data = await res.json();
      chat.innerHTML = data.map(m => `<li>${m.text}</li>`).join("");
    }

    async function send() {
      const text = document.getElementById("msg").value;
      await fetch(`${url}/messages`, {
        method: "POST",
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify({ text })
      });
    }

    setInterval(fetchMessages, 2000); // Poll every 2 seconds
  </script>
</body>
</html>

```

✓ Pros

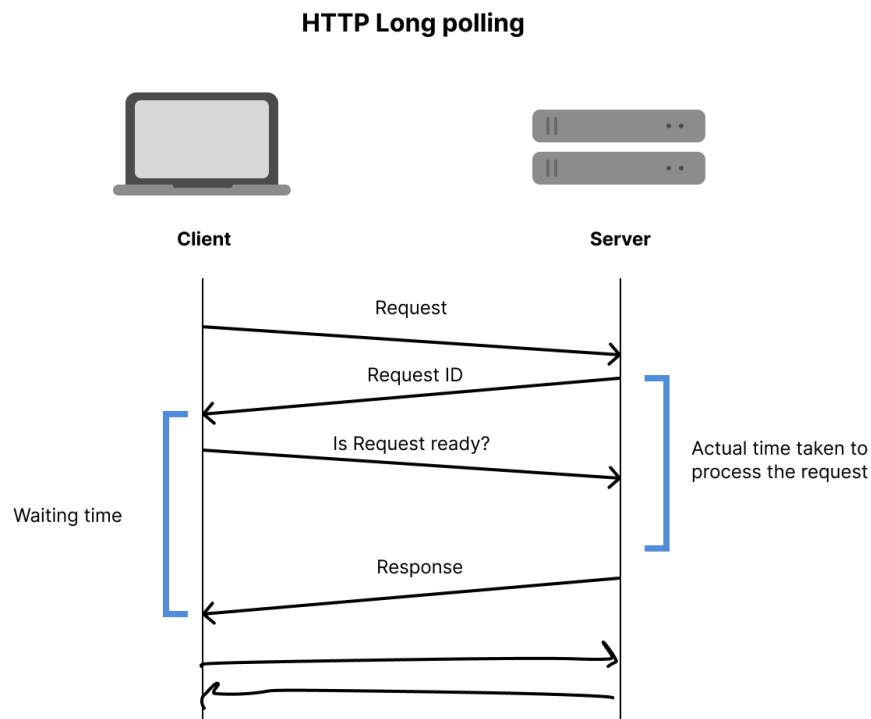
- Easy to understand and implement
- No special protocol required

✗ Cons

- Wastes server resources with repeated calls
- Not truly real-time (you may wait up to X seconds)
- Doesn't scale well with many users

🕒 2. Long Polling

Long polling is smarter. Instead of checking repeatedly, the client sends a request and the server **waits** until there's a new message. Once it has data, it responds, and the client immediately makes another request.



HTTP Long Polling timeline diagram

Long Polling server Node.js:

```
const express = require("express");

const cors = require("cors");

const path = require("path");

const app = express();

app.use(cors());

app.use(express.json());

let messages = [];

let clients = [];

app.get('/', (req, res) => {

  res.sendFile(path.join(__dirname, 'views', 'index.html'));

});

app.get("/messages", (req, res) => {

  // Wait up to 20 seconds for a message

  const timeout = setTimeout(() => res.json([]), 20000);

  clients.push({ res, timeout });

});
```

```

app.post("/messages", (req, res) => {

  const { text } = req.body;

  const message = { text, timestamp: Date.now() };

  messages.push(message);

  // Respond to all waiting clients

  clients.forEach(({ res, timeout }) => {

    clearTimeout(timeout);

    res.json([message]);

  });

  clients = [];

  res.sendStatus(200);

});

app.listen(3002, () => console.log("Long Polling server running on port 3002"));

```

Long Polling client HTML:

```

<!DOCTYPE html>
<html>
<body>
  <h3>Long Polling Chat</h3>
  <input id="msg" placeholder="Type a message" />
  <button onclick="send()">Send</button>
  <ul id="chat"></ul>

  <script>
    const url = "http://localhost:3002";
    const chat = document.getElementById("chat");

    async function poll() {
      const res = await fetch(`${url}/messages`);
    }
  </script>

```

```
const data = await res.json();
data.forEach(m => {
  const li = document.createElement("li");
  li.innerText = m.text;
  chat.appendChild(li);
});
poll(); // Start next long poll
}

async function send() {
  const text = document.getElementById("msg").value;
  await fetch(`${url}/messages`, {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({ text })
  });
}

poll(); // Start first long poll
</script>
</body>
</html>
```

This gives a more “live” feel while avoiding repeated requests.

✓ Pros

- Feels close to real-time
- Reduces unnecessary requests

✗ Cons

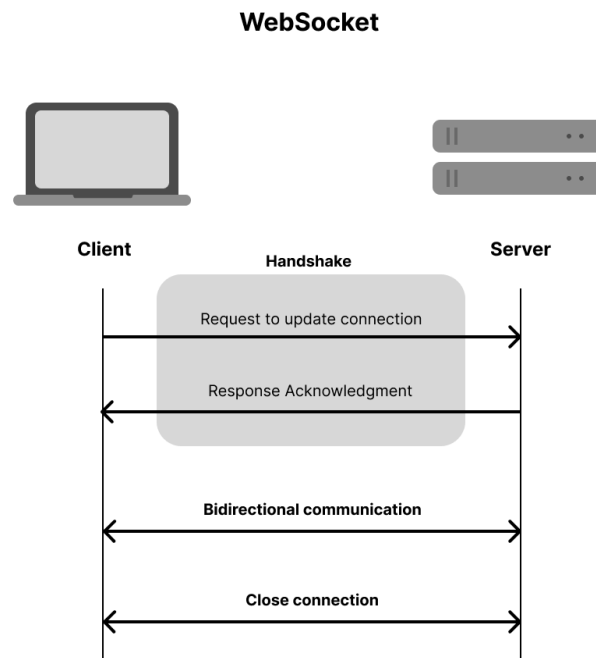
- The server holds connections open, which can be expensive
 - More complex than short polling
 - Still not 100% live
-

3. WebSocket – The Real Solution

Now comes the real-time king: **WebSocket**.

Document: <https://websocket.org/>

WebSocket upgrades a regular HTTP connection into a **persistent, bidirectional channel**. Once the handshake is done, both client and server can talk freely — no more asking for updates!



WebSocket handshake and message flow diagram

WebSocket server Node.js:

```
const express = require("express");
const http = require("http");
const WebSocket = require("ws");
const app = express();
const server = http.createServer(app);
const wss = new WebSocket.Server({ server });

let sockets = [];

wss.on("connection", ws => {
  sockets.push(ws);
});
```

```

ws.on("message", message => {
  sockets.forEach(s => s.send(message)); // Broadcast
});

ws.on("close", () => {
  sockets = sockets.filter(s => s !== ws);
});
});

server.listen(3003, () => console.log("WebSocket server on port 3003"));

```

WebSocket client HTML:

```

<!DOCTYPE html>
<html>
<body>
  <h3>WebSocket Chat</h3>
  <input id="msg" placeholder="Type a message" />
  <button onclick="send()">Send</button>
  <ul id="chat"></ul>

  <script>
    const socket = new WebSocket("ws://localhost:3003");
    const chat = document.getElementById("chat");

    socket.onmessage = async (event) => {
      const li = document.createElement("li");
      // li.innerText = event.data; // It will return you Blog object
      li.innerText = await event.data.text(); // Convert Blog to text
      chat.appendChild(li);
    };

    function send() {
      const text = document.getElementById("msg").value;
      socket.send(text);
    }
  </script>
</body>
</html>

```

It's perfect for chat apps, multiplayer games, collaborative tools, and more.

✓ Pros

- Truly real-time — instant updates
- Lower server load (no repeated requests)
- Supports full duplex communication

✗ Cons

- Needs a bit more setup
- Requires browser/server WebSocket support



Comparison Summary

Method	Real-Time?	Server Load	Complexity
Short Polling	✗ No	● High	● Simple
Long Polling	⚠ Almost	● Medium	● Medium
WebSocket	✓ Yes	● Low	● Simple



Conclusion

We started with a problem — HTTP is great, but not for live messaging. We tried short polling (easy but inefficient), then long polling (closer but not perfect), and finally arrived at WebSocket, which gives us the live experience we truly want.

Winner: WebSocket!